

Selenium with Java

A practical manual: setup, elements, waits, and real project structure

Covers: environment setup (Maven & manual classpath), browser control, locating elements, interacting with elements, waits, dropdowns, alerts, frames/windows, headless mode, and JUnit integration.

Table of Contents

1. What Selenium Is and How It Works
2. Project Setup — Maven (Recommended)
3. Project Setup — Manual Classpath (No Maven)
4. Your First Script: Opening a URL
5. Locating Elements (By)
6. Interacting With Elements
7. Waits — Explicit, Implicit, Fluent
8. Dropdowns (Select)
9. Alerts / Popups
10. Frames and Multiple Windows
11. Headless Mode
12. JUnit 5 Integration
13. Common Errors and Fixes
14. Full Worked Examples
15. Quick Reference Cheat Sheet

1. What Selenium Is and How It Works

Selenium is a browser automation framework. Your Java code talks to a small program called a **driver** (e.g. **chromedriver.exe** for Chrome), which in turn controls a real browser window — clicking buttons, typing text, reading page content — exactly as a human would.

The typical flow in every Selenium script is:

- **Start** a browser session by creating a `WebDriver` object.
- **Navigate** to a URL with `driver.get(url)`.
- **Find** one or more elements on the page (buttons, inputs, links, etc.).
- **Interact** with them — click, type, read text.
- **Close** the browser with `driver.quit()` when done.

Note: Since Selenium 4.6+, you don't need to manually download `chromedriver.exe` — **Selenium Manager** (built into the `selenium-java` library) detects your installed Chrome version and fetches the matching driver automatically the first time you run a script.

2. Project Setup — Maven (Recommended)

Maven is a build tool that automatically downloads and manages libraries (dependencies) for you. This avoids the classpath headaches of manually downloading jar files.

2.1 Folder structure

```
JAVA SELENIUM/  
■■■ pom.xml  
■■■ src/  
    ■■■ main/  
        ■■■ java/  
            ■■■ OpenUrl.java
```

2.2 pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.bhai</groupId>  
  <artifactId>selenium-demo</artifactId>  
  <version>1.0</version>  
  
  <properties>  
    <maven.compiler.source>21</maven.compiler.source>  
    <maven.compiler.target>21</maven.compiler.target>  
  </properties>  
  
  <dependencies>  
    <dependency>  
      <groupId>org.seleniumhq.selenium</groupId>
```

```
<artifactId>selenium-java</artifactId>
<version>4.24.0</version>
</dependency>
</dependencies>
</project>
```

2.3 Running

From the project root (where pom.xml lives):

```
mvn compile
mvn exec:java -Dexec.mainClass="OpenUrl"
```

Note: If `exec:java` isn't recognized, add the `exec-maven-plugin` to your pom.xml's `<build>` section, or simply open the project in an IDE like IntelliJ IDEA / Eclipse and click Run — the IDE handles Maven dependencies automatically with no extra commands.

3. Project Setup — Manual Classpath (No Maven)

If you're not using Maven, you download the Selenium jar files yourself and tell `javac/java` where to find them using the `-cp` (classpath) flag.

3.1 Download

Get the "Selenium Java" zip from <https://www.selenium.dev/downloads/> and extract it into your project folder. You'll end up with a folder full of `.jar` files (e.g. `selenium-java-4.46.0`).

3.2 Compile

```
javac -cp "selenium-java-4.46.0\*" OpenUrl.java
```

3.3 Run

```
java -cp ".;selenium-java-4.46.0\*" OpenUrl
```

Note: On Windows PowerShell, the classpath separator is a semicolon `;` — on Mac/Linux it's a colon `:`. The `.` in the `java` command includes your current folder so it can find the compiled `OpenUrl.class` file. The trailing `\*` wildcard tells Java to load every jar in that folder.

Downsides of this approach: every time you add a new library, you must manually download it and update your classpath. Maven (or an IDE) automates this entirely — worth switching to once you're doing more than one quick script.

4. Your First Script: Opening a URL

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class OpenUrl {
    public static void main(String[] args) {
        // Selenium Manager auto-downloads the matching ChromeDriver
        WebDriver driver = new ChromeDriver();

        driver.get("https://www.google.com");

        System.out.println("Page title: " + driver.getTitle());

        driver.quit(); // always close the browser when done
    }
}
```

`driver.get(url)` navigates to a page and blocks until the page has finished loading. `driver.quit()` closes the browser window and ends the driver process — always call it, or you'll leave orphaned browser processes running.

5. Locating Elements (By)

Before you can click or type into anything, you need to find it on the page. Selenium locates elements using the `By` class.

5.1 Finding the right locator

Right-click any element in Chrome → **Inspect** → look at its HTML attributes. Example:

```
<input id="username" name="user" class="login-input" type="text">
```

You can then locate it several ways (prefer `id` when available — it's the most reliable and specific):

```
driver.findElement(By.id("username"));
driver.findElement(By.name("user"));
driver.findElement(By.className("login-input"));
driver.findElement(By.cssSelector("input.login-input"));
driver.findElement(By.xpath("//input[@id='username']"));
driver.findElement(By.tagName("input"));
driver.findElement(By.linkText("Sign up"));
driver.findElement(By.partialLinkText("Sign"));
```

5.2 Single vs. multiple elements

Method	Returns	Use when
<code>findElement(By...)</code>	<code>WebElement</code>	You expect exactly one match
<code>findElements(By...)</code>	<code>List<WebElement></code>	You expect zero or more matches (e.g. all rows in a table)

```
List<WebElement> rows = driver.findElements(By.cssSelector("table tr"));
System.out.println("Row count: " + rows.size());
for (WebElement row : rows) {
    System.out.println(row.getText());
}
```

6. Interacting With Elements

Once you have a `WebElement`, the core pattern is: **Find** → **Act** → **(optionally) Read**.

```
WebElement el = driver.findElement(By.id("someId"));

el.click();                // click a button / link
el.sendKeys("hello");      // type into a text field
el.clear();                // clear a text field before typing
el.getText();              // read visible text (e.g. of a <div> or <p>)
el.getAttribute("value"); // read an HTML attribute
el.isDisplayed();          // true/false – is it visible on screen?
el.isEnabled();           // true/false – can it be interacted with?
el.isSelected();          // true/false – for checkboxes / radio buttons
el.submit();              // submit the form the element belongs to
```

6.1 Worked example — logging into a form

```
driver.get("https://example.com/login");

WebElement username = driver.findElement(By.id("username"));
username.sendKeys("bhai123");

WebElement password = driver.findElement(By.id("password"));
password.sendKeys("mypassword");

WebElement checkbox = driver.findElement(By.id("terms"));
if (!checkbox.isSelected()) {
    checkbox.click();
}

driver.findElement(By.id("loginBtn")).click();
```

6.2 Sending special keys

```
import org.openqa.selenium.Keys;

WebElement searchBox = driver.findElement(By.name("q"));
searchBox.sendKeys("selenium java");
searchBox.sendKeys(Keys.ENTER);      // press Enter
searchBox.sendKeys(Keys.chord(Keys.CONTROL, "a")); // Ctrl+A
```

7. Waits — Explicit, Implicit, Fluent

Pages load asynchronously — elements often don't exist the instant the page starts loading. Waits tell Selenium to pause until a condition is met, instead of failing immediately or guessing with a fixed delay.

7.1 Explicit wait (preferred)

Waits for a *specific* condition, checking repeatedly until it's true or a timeout is hit.

```
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import java.time.Duration;

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

WebElement result = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("result"))
);
```

7.2 Common ExpectedConditions

```
ExpectedConditions.presenceOfElementLocated(By.id("data"));           // exists in DOM
ExpectedConditions.visibilityOfElementLocated(By.id("data"));         // exists AND visible
ExpectedConditions.elementToBeClickable(By.id("submit"));             // visible AND enabled
ExpectedConditions.invisibilityOfElementLocated(By.id("loader"));     // e.g. spinner gone
ExpectedConditions.titleContains("Dashboard");
ExpectedConditions.alertIsPresent();
```

7.3 Implicit wait (simple fallback)

Applies globally: for every `findElement` call, Selenium will retry for up to N seconds before throwing `NoSuchElementException`.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
```

7.4 Fluent wait (fine-grained control)

Like an explicit wait, but lets you set the polling interval and which exceptions to ignore while waiting.

```
import org.openqa.selenium.support.ui.FluentWait;
import java.util.function.Function;

Wait<WebDriver> fluentWait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(15))
    .pollingEvery(Duration.ofMillis(300))
    .ignoring(NoSuchElementException.class);

WebElement el = fluentWait.until(d -> d.findElement(By.id("result")));
```

Note: Don't mix implicit and explicit waits in the same script — combining them causes unpredictable timing bugs. Pick explicit waits as your default; avoid `Thread.sleep()` entirely except for quick manual debugging.

8. Dropdowns (Select)

Standard HTML `<select>` dropdowns need the `Select` helper class — clicking the raw element won't select an option.

```
import org.openqa.selenium.support.ui.Select;

Select dropdown = new Select(driver.findElement(By.id("country")));

dropdown.selectByVisibleText("India"); // by visible label
dropdown.selectByValue("IN");          // by the <option value="...">
dropdown.selectByIndex(2);              // by position (0-based)

// Reading back
String selected = dropdown.getFirstSelectedOption().getText();

// Multi-select dropdowns
dropdown.deselectAll();
List<WebElement> allOptions = dropdown.getOptions();
```

Note: Custom dropdowns built with `<div>/JavaScript` (common in React/Angular apps) are NOT real `<select>` elements — the `Select` class won't work on them. Instead, click to open the dropdown, then click the desired option as a normal element.

9. Alerts / Popups

JavaScript's native `alert()`, `confirm()`, and `prompt()` dialogs pause page execution and must be handled through Selenium's `Alert` interface — not as a regular `WebElement`.

```
import org.openqa.selenium.Alert;

Alert alert = driver.switchTo().alert();

System.out.println(alert.getText()); // read the message

alert.accept(); // click OK
// alert.dismiss(); // click Cancel
// alert.sendKeys("input text"); // for prompt() dialogs, before accept()
```

10. Frames and Multiple Windows

10.1 iframes

Elements inside an `<iframe>` aren't reachable until you switch context into that frame.

```
driver.switchTo().frame("frameName"); // by name or id attribute
// driver.switchTo().frame(0);         // or by index
// driver.switchTo().frame(driver.findElement(By.tagName("iframe"))); // by element

driver.findElement(By.id("insideFrame")).click();
```

```
driver.switchTo().defaultContent();           // back to the main page
```

10.2 New tabs / windows

```
String originalWindow = driver.getWindowHandle();

// ... action that opens a new tab/window happens here ...

for (String handle : driver.getWindowHandles()) {
    if (!handle.equals(originalWindow)) {
        driver.switchTo().window(handle);
        break;
    }
}

// do things in the new window...

driver.close();           // close just this window
driver.switchTo().window(originalWindow); // go back to the original
```

11. Headless Mode

Headless mode runs the browser with no visible UI — useful for CI pipelines, servers, or background scripts where you don't need to watch the browser.

```
import org.openqa.selenium.chrome.ChromeOptions;

ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");
options.addArguments("--window-size=1920,1080");
options.addArguments("--disable-gpu");

WebDriver driver = new ChromeDriver(options);
```

Note: Some sites detect and block headless browsers. If a script that works normally starts failing only in headless mode, that's usually the cause — try adjusting the user-agent or window size, or run headed while debugging.

12. JUnit 5 Integration

For real test suites, wrap Selenium in JUnit so tests run repeatably with proper setup/teardown and assertions.

12.1 Dependency (pom.xml)

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.11.0</version>
  <scope>test</scope>
</dependency>
```

12.2 Test class

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class LoginTest {
    WebDriver driver;

    @BeforeEach
    void setup() {
        driver = new ChromeDriver();
    }

    @Test
    void testPageTitle() {
        driver.get("https://example.com");
        assertEquals("Example Domain", driver.getTitle());
    }

    @Test
    void testLoginFormPresent() {
```

```

        driver.get("https://example.com/login");
        assertTrue(driver.findElement(By.id("loginBtn")).isDisplayed());
    }

    @AfterEach
    void teardown() {
        driver.quit();
    }
}

```

13. Common Errors and Fixes

Error	Cause	Fix
package org.openqa.selenium does not exist	Selenium jars not on the classpath	Use Maven, or add -cp "path*" to javac/java
NoSuchElementException	Locator is wrong, or element hasn't loaded yet	Verify the locator in DevTools; add an explicit wait
StaleElementReferenceException	DOM changed after you grabbed the reference	Re-find the element right before using it again
ElementNotInteractableException	Element is hidden, disabled, or covered by another element	Wait for elementToBeClickable; scroll into view
ElementClickInterceptedException	Another element (e.g. a modal/overlay) is on top	Close the overlay first, or scroll/wait before clicking
SessionNotCreatedException	Installed Chrome version and driver version mismatch	Update Chrome; Selenium Manager usually auto-fixes this

Other things to double check when a script fails:

- Always call `driver.quit()` in a finally block (or `@AfterEach` in tests) — leftover browser/driver processes cause weird failures on the next run.
- Never mix implicit and explicit waits.
- If you switched into an iframe and can't find an element you expect on the main page, you probably forgot `driver.switchTo().defaultContent()`.

14. Full Worked Examples

Each example below is a complete, runnable Java class — copy the whole thing into its own file (matching the class name) and run it as shown earlier in Sections 2–3.

14.1 Example: Google Search Automation

Opens Google, types a query, presses Enter, and prints the resulting page title.

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;

public class GoogleSearchExample {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        try {
            driver.get("https://www.google.com");

            WebElement searchBox = wait.until(
                ExpectedConditions.visibilityOfElementLocated(By.name("q"))
            );
            searchBox.sendKeys("Selenium Java tutorial");
            searchBox.sendKeys(Keys.ENTER);

            wait.until(ExpectedConditions.titleContains("Selenium Java"));
            System.out.println("Result page title: " + driver.getTitle());
        } finally {
            driver.quit();
        }
    }
}
```

14.2 Example: Login Form With Explicit Waits

A realistic login flow — waits for the button to become clickable rather than assuming the page has already finished loading.

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;

public class LoginExample {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
```

```

        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        try {
            driver.get("https://example.com/login");

            wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")))
        ))
            .sendKeys("bhai123");
            driver.findElement(By.id("password")).sendKeys("mypassword");

            WebElement loginBtn = wait.until(
                ExpectedConditions.elementToBeClickable(By.id("loginBtn"))
            );
            loginBtn.click();

            WebElement welcome = wait.until(
                ExpectedConditions.visibilityOfElementLocated(By.id("welcomeMessage"))
            );
            System.out.println("Logged in. Message: " + welcome.getText());
        } catch (TimeoutException e) {
            System.out.println("Login failed or page took too long: " + e.getMessage()
        );
        } finally {
            driver.quit();
        }
    }
}

```

14.3 Example: Scraping a List of Elements

Finds every matching element on a page and prints its text — useful for pulling product names, headlines, prices, etc.

```

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import java.util.List;

public class ScrapeListExample {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();

        try {
            driver.get("https://example.com/products");

            List<WebElement> productNames =
                driver.findElements(By.cssSelector(".product-card h2"));

            System.out.println("Found " + productNames.size() + " products:");
            for (WebElement product : productNames) {
                System.out.println("- " + product.getText());
            }
        } finally {
            driver.quit();
        }
    }
}

```

```
}
```

14.4 Example: Dropdown Selection

Selects a country from a standard HTML <select> dropdown and confirms the choice.

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.Select;

public class DropdownExample {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();

        try {
            driver.get("https://example.com/signup");

            Select countryDropdown = new Select(driver.findElement(By.id("country")));
            countryDropdown.selectByVisibleText("India");

            String selected = countryDropdown.getFirstSelectedOption().getText();
            System.out.println("Selected country: " + selected);
        } finally {
            driver.quit();
        }
    }
}
```

14.5 Example: Taking a Screenshot

Captures the current browser view and saves it as a PNG file — handy for debugging or logging test failures.

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import java.io.File;
import java.nio.file.Files;

public class ScreenshotExample {
    public static void main(String[] args) throws Exception {
        WebDriver driver = new ChromeDriver();

        try {
            driver.get("https://example.com");

            TakesScreenshot ts = (TakesScreenshot) driver;
            File screenshot = ts.getScreenshotAs(OutputType.FILE);
            Files.copy(screenshot.toPath(), new File("page.png").toPath());

            System.out.println("Screenshot saved as page.png");
        } finally {
            driver.quit();
        }
    }
}
```

14.6 Example: Running JavaScript Directly

Sometimes clicking or scrolling through Selenium isn't reliable (e.g. an element hidden behind a sticky header). `JavaScriptExecutor` lets you run raw JS in the page context.

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;

public class JavaScriptExecutorExample {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();

        try {
            driver.get("https://example.com");

            JavaScriptExecutor js = (JavaScriptExecutor) driver;

            // Scroll an element into view
            WebElement footer = driver.findElement(By.tagName("footer"));
            js.executeScript("arguments[0].scrollIntoView(true);", footer);

            // Read a value directly from the DOM
            String title = (String) js.executeScript("return document.title;");
            System.out.println("Title via JS: " + title);

            // Force-click an element JS can reach even if Selenium says it's not clic
            kable
            WebElement hiddenBtn = driver.findElement(By.id("hiddenBtn"));
            js.executeScript("arguments[0].click();", hiddenBtn);
        } finally {
            driver.quit();
        }
    }
}
```

14.7 Example: Hover and Drag-and-Drop (Actions class)

The `Actions` class handles mouse movements that a plain `.click()` can't — hovering over a menu, or dragging one element onto another.

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

public class ActionsExample {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        Actions actions = new Actions(driver);

        try {
            driver.get("https://example.com/menu");

            // Hover to reveal a submenu
            WebElement menuItem = driver.findElement(By.id("menuParent"));
            actions.moveToElement(menuItem).perform();
        }
    }
}
```



```
        WebElement subMenuLink = driver.findElement(By.id("subMenuLink"));
        subMenuLink.click();

        // Drag and drop
        WebElement source = driver.findElement(By.id("draggable"));
        WebElement target = driver.findElement(By.id("dropZone"));
        actions.dragAndDrop(source, target).perform();
    } finally {
        driver.quit();
    }
}
```

15. Quick Reference Cheat Sheet

Setup

```
WebDriver driver = new ChromeDriver();
driver.get("https://example.com");
driver.quit();
```

Locators

```
By.id("x")    By.name("x")    By.className("x")    By.cssSelector("x")
By.xpath("//tag[@attr='x']")    By.tagName("x")    By.linkText("x")
```

Actions

```
el.click(); el.sendKeys("text"); el.clear(); el.submit();
el.getText(); el.getAttribute("href");
el.isDisplayed(); el.isEnabled(); el.isSelected();
```

Waits

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("x")));
wait.until(ExpectedConditions.elementToBeClickable(By.id("x")));
```

Dropdowns / Alerts / Frames

```
new Select(el).selectByVisibleText("India");
driver.switchTo().alert().accept();
driver.switchTo().frame("frameName");
driver.switchTo().defaultContent();
```

Headless

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");
WebDriver driver = new ChromeDriver(options);
```

End of manual. For deeper topics (Actions class for drag-and-drop/hover, JavaScriptExecutor, screenshots, Page Object Model design, or parallel test execution), just ask.